inalco
Institut national des langues et civilisations orientales

Contrôle 1 : Techniques du Web

Documents et ordinateur non autorisés.

Nom :

Prénom :

Nº Étudiant :

Note : **/ 20**

I. Questions de compréhension de code (/ 8 points)

Lisez attentivement les fragments de code Python suivants et répondez avec précision aux questions associées.

1) Manipulation de fichiers et sérialisation JSON (2 points)

```python
import json
from pathlib import Path

chemin = Path("/home/user/documents/file.json")
mon_objet = {"name": "Alice", "age": 30, "prof": False}

with open(chemin, "w") as f:
    json.dump(mon_objet, f)
```

a) Quel est le type précis des variables `chemin`, `mon_objet` et `f` ?

b) À quoi sert la fonction `json.dump` ? Donnez sa signature (arguments d'entrée et valeur de retour).

c) Quelle fonction de la bibliothèque `json` permet de charger le contenu d'un fichier .json en un objet Python natif ?

2) Gestion des exceptions et journalisation (Logging) (2 points)

```python
import logging

def diviser_par_contenu(chemin_fichier):
    logging.debug("Lecture du fichier")

    try:
        with open(chemin_fichier, "r") as fichier:
            contenu = fichier.read()
            valeur = int(contenu)
            resultat = 100 / valeur
    except ValueError:
        logging.error("Erreur de conversion")

    return resultat
```

a) Que fait ce code ? Quel problème surviendra à l'exécution si le fichier ne contient pas un entier valide ?

b) Que faudrait-il ajouter à la ligne 13 pour que, **dans tous les cas**, le programme affiche à l'écran le chemin du fichier traité ?

c) Expliquez la différence d'usage et de niveau entre `logging.debug` et `logging.error`.

3) Moissonnage Web et extraction HTML (Scraping) (2 points)

```python
import urllib.request
from bs4 import BeautifulSoup

url = "https://www.exemple.com/profils"
requete = urllib.request.Request(url, headers={"User-Agent": "Etudiant INALCO"})

with urllib.request.urlopen(requete) as page:
    html = page.read()

soup = BeautifulSoup(html, "html.parser")
balises_age = soup.find_all("age")

for balise in balises_age:
    texte = balise.get_text()
    print(texte)
```

a) A quoi sert ce code ? Quel est le nom technique donné à cette pratique ?

b) À quoi sert précisément la méthode `get_text()` dans la boucle finale ?

4) Syntaxe idiomatique Python (« Pythonista ») (2 points)

```python
def formater_mots(separateur, *mots):
    elements = []
    for i in range(len(mots)):
        elements.append(str(i) + "-" + mots[i].upper())
    return separateur.join(elements)

mots_cles = ["python", "web", "inalco"]
resultat = formater_mots(" ~ ", *mots_cles)
print(resultat)
```

a) Exécutez le code mentalement. Que vaut exactement la variable `resultat` affichée à l'écran ?

b) Expliquez le rôle de la méthode `str.join()` ainsi que la signification de l'astérisque dans la signature `*mots` et lors de l'appel `*mots_cles`.

c) Comment pourrait-on réécrire les lignes de la boucle `for` (lignes 3 et 4) de manière plus efficace et idiomatique pour éviter d'écrire `mots[i]` ?

II. Data Science enfumée (/ 6 points)

Voici un extrait d'un *DataFrame* Pandas nommé `df` présentant le prix moyen des paquets de cigarettes dans différents pays d'Europe :

Pays	Marque	Prix 2025 (EUR)	Prix 2026 (EUR)	Prix fournisseur (EUR)
France	Marlboro	12.50	13.00	1.50
France	Camel	12.00	12.50	1.40
Suisse	Marlboro	10.20	10.50	1.50
Suisse	Camel	9.90	10.20	1.40
Allemagne	Marlboro	8.40	8.70	1.50
Allemagne	Camel	8.20	8.50	1.40

Consigne générale pour le QCM (page suivante) : Répondez aux 6 questions sur la page 3 en vous référant au tableau ci-dessus et au graphique. Justifiez brièvement vos choix. **Attention, plusieurs réponses peuvent être correctes par question.**

Question 1 : Quelles instructions permettent d'obtenir les dimensions exactes du dataset (lignes et colonnes) ?

☐ a) `df.size` ☐ b) `df.shape`
☐ c) `len(df)` ☐ d) `df.info()`

Question 2 : Quelles commandes permettent d'extraire le prix minimum des cigarettes pour l'année 2025 ?

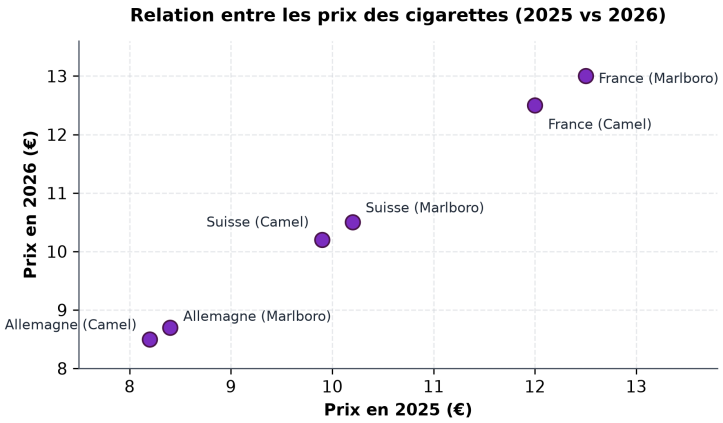
☐ a) `df.min('Prix_2026')` ☐ b) `df['Prix_2025'].min()`
☐ c) `df['Prix_2025'].max()` ☐ d) `min(df['Prix_2025'])`

Question 3 : Vous souhaitez calculer le taux d'évolution du prix entre 2025 et 2026. Quelle(s) ligne(s) font cela ?

☐ a) `df['Evolution'] = df['Prix_2026'] / df['Prix_2025']`
☐ b) `df['Evolution'] = (df['Prix_2026'] - df['Prix_2025']) / df['Prix_2025']`
☐ c) `df['Prix_2026'] - df['Prix_2025']`
☐ d) `df['Prix_2025'] / df['Prix_2026']`

Question 4 : Que fait exactement la ligne suivante : `df.groupby('Marque')['Prix_2026'].mean()` **?**

☐ a) Remplace les valeurs de `Prix_2026` par le prix moyen de chaque marque.
☐ b) Crée une vue regroupant par marque et calcule la moyenne de leurs prix en 2026.
☐ c) Renvoie une `Series` indexée par marque avec le prix moyen 2026.
☐ d) Calcule la moyenne globale des prix en 2026 sans distinction de marque.



Question 5 : Quelle commande Matplotlib permet de générer fidèlement ce type de graphique ?

☐ a) `plt.hist(df['Prix_2025'])`
☐ b) `plt.scatter(df['Prix_2025'], df['Prix_2026'])`
☐ c) `plt.plot(df['Prix_2025'], df['Prix_2026'])`
☐ d) `plt.bar(df['Prix_2025'], df['Prix_2026'])`

Question 6 : À quoi peut concrètement servir ce graphique (nuage de points) dans notre analyse ?

☐ a) À visualiser la répartition globale des prix sur une seule année.
☐ b) À identifier visuellement s'il existe une corrélation forte entre le prix 2025 et le prix 2026.
☐ c) À repérer rapidement d'éventuelles valeurs aberrantes ou des augmentations atypiques.
☐ d) À représenter l'évolution continue et journalière des prix sur les deux années.

III. Étude de mots et algorithmique (/ 8 points)

On considère l'algorithme Python ci-dessous composé de deux fonctions :

```python
def is_palindrome(substring):
    if len(substring) < 2:
        return True

    for i in range(len(substring) // 2):
        if substring[i] == substring[-(i+1)]:
            return True

    return False

def longestPalindrome(s):
    longest = 0
    longest_substring = ""

    for i in range(len(s) + 1):
        for j in range(i, len(s) + 1):
            substring = s[i:j]

            if is_palindrome(substring):
                if len(substring) > longest:
                    longest = len(substring)
                    longest_substring = substring

    print(longest_substring)
    return longest_substring
```

a) Rôle général : Expliquez ce que cet algorithme est censé faire (en précisant le type de l'entrée et de la sortie) en une phrase concise.

b) Déroulement pas à pas : Exécutez l'algorithme à la main avec la chaîne `"caracas"`. Vous pouvez synthétiser les principales étapes dans un tableau (valeurs de `i`, `j`, `substring`, résultat du test de palindrome et mise à jour de `longest_substring`).
Que constatez-vous ?

c) Détection et correction d'erreur : Une erreur s'est glissée dans l'algorithme. Où se situe-t-elle exactement dans le code, et comment la corriger ?

d) Syntaxe Python : Comment pourrait-on réécrire les deux affectations des lignes 21 et 22 en une seule ligne ?

e) Complexité : Quelle est la complexité temporelle théorique dans le pire des cas (en notation $\mathcal{O}$) de cet algorithme en fonction de la longueur N de la chaîne s ?

f) Modification de code : Modifiez le code de la fonction `longestPalindrome` pour qu'elle ne cherche plus seulement le plus long palindrome, mais renvoie la **liste de tous les sous-palindromes distincts** contenus dans le mot, pourvu qu'ils comportent strictement plus de 3 lettres.

g) Un autre algorithme : Quelle modification minimale faudrait-il apporter au code pour qu'il retourne tous les sous-mots qui commencent et se terminent par la même lettre, peu importe leur contenu intérieur ?